

Workforce_Summary

February 25, 2026

1 Workforce System — Summary & Housekeeping Ops

Project path: ~/scheduler/workforce

Stack: FastAPI · SQLAlchemy 2 · Alembic · SQLite (dev) / Postgres (prod)

Deployed on: PythonAnywhere (WSGI via a2wsgi)

1.1 1. System Overview

The **Cloud Workforce System** (v0.2.0) is a multi-tenant workforce scheduling API. Each **Business** can have multiple **Locations**, hire **Employees** into **Employments**, assign **Roles**, publish **Shifts**, and manage the full staffing lifecycle.

1.1.1 Architecture — two API planes

Plane	Prefix	Purpose
Legacy MVP	/	Original routes: businesses, employees, shifts, training, audit
V1 Control	/api/v1/	Auth, RBAC, tenant management, worker self-service, marketplace, scheduling agent, timeclock, messaging, dashboard

1.1.2 V1 route modules

Module	Description
auth	JWT login, refresh, logout
control	Super-admin: create tenants, users, assign roles
tenant	Business-admin: manage employees, locations, roles, permissions
worker	Worker self-service: profile, availability, assignments
agent	AI scheduling agent runs (Ollama / rule-based fallback)
timeclock	Clock-in / clock-out, timecard management
marketplace	Job postings, shift requests, shift swap requests, swap rules
schedule	Schedule rules, schedule shifts, draft assignments

Module	Description
dashboard	Widgets for control, tenant, and worker dashboards
messaging	Channels, messages (48 h TTL), WebSocket, external API keys

1.1.3 Core design choices

- **UUID String(36) PKs** — portable across SQLite and Postgres.
- **Tenant safety** — every employee/shift query joins through `Employment.business_id`.
- **Audit middleware** — `AuditMiddleware` writes an `AuditEvent` row for every successful mutation.
- **render_as_batch=True** in Alembic for SQLite ALTER TABLE compatibility.
- **WSGI bridge** — `a2wsgi.ASGIMiddleware` wraps the ASGI FastAPI app for PythonAnywhere.

1.2 2. Data Models

Domain file	Key models
business.py	Business, Location
employee.py	Employee, Employment, EmployeeRole, AvailabilityBlock
scheduling.py	Shift, ShiftAssignment
training.py	TrainingModule, EmployeeTraining
identity.py	User, RefreshToken, Membership, WorkerProfile, BizRole, Permission, Agent, AgentRun, AuditEvent, WorkerAvailability
marketplace.py	JobPosting, ShiftRequest, TrainingRequest, ShiftSwapRequest, SwapPermissionRule
timeclock.py	TimeclockEntry, Timecard
schedule.py	ScheduleRule, ScheduleShift, ScheduleAssignment
dashboard.py	DashboardWidget
messaging.py	Channel, ChannelMember, Message, MessagingApiKey
audit.py	AuditLog (legacy)

1.3 3. Environment Setup

```
cd ~/scheduler/workforce
source venv/bin/activate
pip install -e .
```

Key `.env` variables:

Variable	Default	Notes
DATABASE_URL	sqlite:///./dev.db	Use absolute path on PythonAnywhere
ENV	dev	Set prod in production
SECRET_KEY	placeholder	Change in production
ACCESS_TOKEN_EXPIRE_MINUTES	60	JWT access token lifetime
REFRESH_TOKEN_EXPIRE_DAYS	7	Refresh token lifetime

1.4 4. Housekeeping Operations

1.4.1 4.1 Database migrations

```
cd ~/scheduler/workforce && source venv/bin/activate
```

```
# Apply all pending migrations
alembic upgrade head
```

```
# Check current migration revision
alembic current
```

```
# View migration history
alembic history --verbose
```

```
# Autogenerate a new migration after model changes
alembic revision --autogenerate -m "describe_your_change"
```

Note: When adding a new ORM model, import it in `app/models/__init__.py` before running `autogenerate`.

1.4.2 4.2 Seed demo data

```
python -m app.cli.main seed-demo
# Prints a Shift UUID - save it for testing the candidates endpoint
```

1.4.3 4.3 Run tests

```
pytest -q                                # all tests
pytest tests/test_matching.py -v          # matching tests only
pytest tests/test_matching.py::test_preferred_comes_first -v # single test
```

Tests use an **in-memory SQLite** database (`DATABASE_URL=sqlite://`) — no `dev.db` contamination.

1.4.4 4.4 Start the dev server

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
# Interactive docs: http://localhost:8000/docs
```

1.4.5 4.5 PythonAnywhere reload

After any code or migration change on PythonAnywhere: 1. Run `alembic upgrade head` in a Bash console. 2. Click **Reload** on the Web tab. 3. Verify: `curl https://<username>.pythonanywhere.com/health`

1.4.6 4.6 Expired messaging cleanup

Messages have a **48-hour TTL** (`expires_at` column).

The route layer calls `_purge_expired(db)` inline — no separate cron needed for dev.

In production, consider a scheduled task or cron if message volume is high:

One-off purge via Python

```
cd ~/scheduler/workforce && source venv/bin/activate
python - <<'EOF'
from datetime import datetime, timezone
from app.core.db import db_session
from app.models.messaging import Message
from sqlalchemy import delete

with db_session() as db:
    deleted = db.execute(
        delete(Message).where(Message.expires_at < datetime.now(timezone.utc))
    )
    print(f"Deleted {deleted.rowcount} expired messages")
EOF
```

1.4.7 4.7 Refresh token housekeeping

Refresh tokens are stored hashed in the `refresh_tokens` table.

Revoked/expired tokens accumulate over time. Purge periodically:

```
python - <<'EOF'
from datetime import datetime, timezone
from app.core.db import db_session
from app.models.identity import RefreshToken
from sqlalchemy import delete

with db_session() as db:
    deleted = db.execute(
        delete(RefreshToken).where(
            (RefreshToken.revoked == True) |
            (RefreshToken.expires_at < datetime.now(timezone.utc))
        )
    )
    print(f"Deleted {deleted.rowcount} stale refresh tokens")
EOF
```

1.4.8 4.8 Audit event review

Every non-GET 2xx response writes an `AuditEvent` with actor info and correlation ID.

```
python - <<'EOF'
from app.core.db import db_session
from app.models.identity import AuditEvent
from sqlalchemy import select

with db_session() as db:
    events = db.scalars(select(AuditEvent).order_by(AuditEvent.created_at.desc()).limit(20)).all()
    for e in events:
        print(e.created_at, e.actor_type, e.actor_id, e.action, e.resource_type, e.resource_id)
EOF
```

1.4.9 4.9 Check database size (SQLite)

```
ls -lh ~/scheduler/workforce/dev.db
sqlite3 ~/scheduler/workforce/dev.db "SELECT name, COUNT(*) FROM sqlite_master JOIN (SELECT tbl_name FROM pragma_table_info(1)) ON tbl_name = name;"
```

1.4.10 4.10 Permissions & roles seed

Default permissions and roles are seeded automatically on every app startup via `on_startup`.
To re-seed manually:

```
python - <<'EOF'
from app.core.db import db_session
from app.services.roles_seed import seed_permissions_and_roles

with db_session() as db:
    seed_permissions_and_roles(db)
    print("Permissions and roles seeded.")
EOF
```

1.5 5. Key Service: Shift Candidate Matching

`app/services/matching.py` — `find_candidates_for_shift(session, shift_id)`

A candidate is eligible if they satisfy **all** of:

1. **Active employment** in the shift's `business_id`.
2. **Correct role** (`EmployeeRole`) within that employment.
3. **Non-unavailable availability** (`AvailabilityBlock`) that fully covers the shift window.
4. **No blocking overlap** — no existing offered/assigned/checked_in assignment on a concurrent shift.

Results are sorted: preferred availability → available, then proficiency DESC, then last_name ASC.

1.6 6. Marketplace & Shift Swap Flow

pending

```

    awaiting_peer    (initiator approved, peer must respond)
        approved
        denied
        pending_coverage
            completed    (tenant calls arrange-coverage → new JobPosting)

```

SwapPermissionRule evaluates by **priority (highest wins)**: - allow → auto-approve without manager review. - deny → block the swap entirely.

1.7 7. Messaging System Notes

- **48-hour message TTL** — expires_at is set on creation; expired rows are purged inline.
- **WebSocket** at /ws/{channel_id}?token=<jwt> — in-memory connection registry, not persistent.
- **Channel types**: announcement (admin-post only), group (any member), direct (2 members).
- **External API keys** stored as SHA-256(raw_key); scoped to optional channel list; passed via X-API-Key header.

1.8 8. Quick Reference — Common Commands

```
cd ~/scheduler/workforce && source venv/bin/activate
```

```
# Migrate
```

```
alembic upgrade head
```

```
# Seed
```

```
python -m app.cli.main seed-demo
```

```
# Match candidates
```

```
python -m app.cli.main match --shift-id <UUID>
```

```
# Tests
```

```
pytest -q
```

```
# Dev server
```

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

```
# Health check
```

```
curl http://localhost:8000/health
```

```
# Candidates endpoint
```

```
curl http://localhost:8000/shifts/<UUID>/candidates
```

```
[ ]: # Health check - run this cell to verify the app is up
import subprocess, json
result = subprocess.run(["curl", "-s", "http://localhost:8000/health"],
    ↪ capture_output=True, text=True)
```

```
print(json.dumps(json.loads(result.stdout), indent=2) if result.stdout else
↳ "Server not running")
```

```
[ ]: # Row counts for all tables
import sys
sys.path.insert(0, "/home/hn3t/scheduler/workforce")

import os
os.environ.setdefault("DATABASE_URL", "sqlite:///home/hn3t/scheduler/workforce/
↳ dev.db")

from sqlalchemy import inspect, text
from app.core.db import engine

with engine.connect() as conn:
    insp = inspect(engine)
    for tbl in sorted(insp.get_table_names()):
        count = conn.execute(text(f'SELECT COUNT(*) FROM "{tbl}"')).scalar()
        print(f"{tbl:<40} {count:>6} rows")
```

```
[ ]: # Purge expired messages
from datetime import datetime, timezone
from app.core.db import db_session
from app.models.messaging import Message
from sqlalchemy import delete

with db_session() as db:
    result = db.execute(delete(Message).where(Message.expires_at < datetime.
↳ now(timezone.utc)))
    print(f"Purged {result.rowcount} expired messages")
```

```
[ ]: # Purge revoked / expired refresh tokens
from app.core.db import db_session
from app.models.identity import RefreshToken
from sqlalchemy import delete, or_

with db_session() as db:
    result = db.execute(
        delete(RefreshToken).where(
            or_(RefreshToken.revoked == True,
                RefreshToken.expires_at < datetime.now(timezone.utc))
        )
    )
    print(f"Purged {result.rowcount} stale refresh tokens")
```

```
[ ]: # Recent audit events (last 20)
from app.core.db import db_session
from app.models.identity import AuditEvent
from sqlalchemy import select

with db_session() as db:
    events = db.scalars(
        select(AuditEvent).order_by(AuditEvent.created_at.desc()).limit(20)
    ).all()
    for e in events:
        print(f"{str(e.created_at)[:19]} {e.actor_type or '-':<8} {e.action:
↵<30} {e.resource_type or '-'}/{e.resource_id or '-'}")
```